Case Study: Senior Full Stack Shopify Engineer

Deliverables: GitHub repo + Loom walkthrough

The Task

Post-purchase upsell flows are a core revenue lever in DTC ecommerce. This project asks you to build one from scratch on Shopify — standing up a dev store, wiring into the checkout lifecycle, and adding an intelligent offer layer. It's a realistic slice of the kind of work this role owns day-to-day.

What to Build

1. Shopify Dev Store

Create a free Shopify development store via a Partner account and seed it with product data sufficient to demonstrate your app's offer logic.

→ Partner signup: partners.shopify.com/signup

2. Post-Purchase Offer App

Build a Shopify app that:

- Reads the contents of a completed checkout/order
- Determines a relevant upsell or cross-sell offer based on order context
- Presents that offer via a post-purchase checkout extension

Post-purchase extensions work without restrictions on dev stores. Use Shopify CLI (shopify app dev) — it auto-configures cart permalinks and handles the post-purchase redirect.

→ Dev docs: About product offers

3. Intelligent Offer Selection

The logic for *which* offer to show should go beyond simple rules. Use an LLM or similar approach to analyze order contents and select or generate an offer. For example: given a cart of gummy vitamins, the system might surface a complementary product with a personalized message explaining why it pairs well.

4. Backend + Analytics

Include a lightweight backend component that:

- Stores offer impressions and acceptance/rejection events
- Exposes a simple dashboard or API endpoint showing offer performance (conversion rate, revenue impact, top-performing offers)

Testing Your Work

Use the Bogus Gateway to place free test orders — no paid plan required on a dev store. You won't need more than a handful of test orders to demonstrate the flow.

→ Test orders: Activating a payment gateway in test mode

Evaluation Criteria

Area	What We're Looking For
Shopify Fluency	Comfort with the ecosystem — dev store setup, APIs, extension points, data model
Full-Stack Execution	Clean separation of concerns, production-minded structure, error handling
Revenue Thinking	Does the offer logic feel like it would move the needle? Thinking about conversion, not just code
AI Integration	Thoughtful use — genuinely improving offer quality, not shoehorned in
Systems Design	How you'd scale beyond a POC. Even if not built, we want to see the thinking

Deliverables

GitHub Repository with a README covering:

- Setup instructions (we will run this locally against your dev store or our own)
- Architecture overview and key decisions
- What you'd do differently with more time

Loom Video walking through:

- The app in action end-to-end
- One or two architectural decisions you're proud of
- How you'd evolve this into a production system

Guidelines

- Use whatever stack you're most productive in. We care about quality, not stack matching.
- You may use AI tools (Cursor, Copilot, Claude, etc.) — we do too. Be prepared to speak to every line of your code.
- Don't over-polish the UI. Clean and functional beats pretty.
- If you hit a wall with the post-purchase extension APIs, it's fine to simulate the experience with a standalone page that receives order data. Tell us what you ran into.
- Cut scope ruthlessly and tell us what you cut and why. We're not looking for a production-ready system — we're looking for signal on how you think, build, and operate as a technical owner.